FULL-STACK DEVELOPER PORTFOLIO

Portfolio

두 개의 풀스택 프로젝트를 통한 성장 기록

PROJECT 01

AuRa

향수 이커머스
4인 팀 프로젝트 · 3주

PROJECT 02

KISETSU

도매 패션 이커머스
1인 솔로 프로젝트 · 2주

React | Node.js | Express | MariaDB | JWT | Socket.IO | REST API | Vite

2025 — 2026

INDEX

목차

PROJECT 01 — AURA 향수 쇼핑몰

01 **프로젝트 개요** 03
AuRa 소개, 기술 스택, 팀 구성

02 **주요 기여 — DB 설계 · 인증 · 장바구니 · 카테고리** 04
담당 기능 상세, 코드 구현 방식

03 **기술적 어필 포인트** 05
관심사 분리, 중복 방지 로직, 서버단 검증

PROJECT 02 — KISETSU 도매 패션 이커머스

04 **프로젝트 개요 · 아키텍처** 06
기획 배경, 시스템 구조, 기술 스택

05 **주요 기능 구현** 07
JWT 인증, 실시간 채팅, 전역 상태, 인기검색어

06 **결과 · 회고** 08
구현 성과, 배운 점, 성장 스토리

PROJECT 01

AuRa

향수 이커머스 쇼핑몰
REST API · 풀스택 팀 프로젝트

유형	기간	역할
4인 팀 프로젝트	약 3주	DB · 인증 · 장바구니 · 카테고리

MariaDB · Node.js · Express · React · Vite · REST API · 커넥션 풀

01

프로젝트 개요

AuRa 향수 쇼핑몰 — 팀 프로젝트

4인	3주	4개	REST
팀 구성	개발 기간	담당 기능	API 설계

서비스 전체 기능

- 회원가입 / 로그인
- 상품 목록 / 상세 / 검색
- 카테고리 필터 & 정렬
- 장바구니 / 위시리스트
- 주문 / 결제 처리
- AI 챗봇 상담
- 미니게임 이벤트

내 담당 기능

- **DB 설계** — MariaDB 커넥션 풀 모듈화
- **로그인 / 회원가입** — REST API, 서버단 검증
- **장바구니** — CRUD API, 중복 방지 로직
- **카테고리** — DB 기반 동적 카테고리, 필터 & 정렬

기술 스택

Database	Backend	Frontend
MariaDB	Node.js, Express	React, Vite

02 주요 기여 — 구현 상세

DB 설계 · 로그인/회원가입 · 장바구니 · 카테고리

DB — 커넥션 풀 모듈화

db.js

```
// MariaDB 커넥션 풀 모듈 분리
const mariadb = require('mariadb')
require('dotenv').config()

const pool = mariadb.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  connectionLimit: 10
})

module.exports = pool
```

로그인 / 회원가입 API

auth.js

```
// POST /api/auth/login
app.post("/api/auth/login", async (req, res) => {
  const { username, password } = req.body
  // 입력값 누락 시 서버단 검증
  if (!username || !password)
    return res.status(400).json({ msg: "필수값 누락" })
  const [user] = await db.query(
    `SELECT * FROM users
     WHERE username=? AND password=?`,
    [username, password]
  )
  if (!user) return res.status(401).json({...})
  res.json({ success: true, user })
})
```

장바구니 — 중복 방지 핵심 로직

cart.js

```
// POST /api/cart/add
// 핵심: 중복 상품 서버단 검증
app.post("/api/cart/add", async (req, res) => {
  const { userId, productId } = req.body
  const [exist] = await db.query(
    `SELECT * FROM cart
     WHERE user_id=? AND product_id=?`,
    [userId, productId]
  )
  if (exist)
    return res.json({ msg: "이미 담긴 상품" })
  await db.query(
    `INSERT INTO cart VALUES(?,?)`,
    [userId, productId]
  )
})
```

카테고리 — DB 기반 동적 쿼리

category.js

```
// GET /api/category (DB 기반)
app.get("/api/category", async (req, res) => {
  // 하드코딩 없이 DB에서 동적 조회
  const cats = await db.query(
    `SELECT * FROM categories`
  )
  res.json(cats)
})
// 가격 필터 + 정렬 동적 쿼리
let sql = `SELECT * FROM products
  WHERE category_id=?`
if (minPrice) sql += ` AND price≥?`
sql += ` ORDER BY \${sortField}`
```

03

기술적 어필 & 회고

설계 철학과 팀 프로젝트에서의 성장

관심사 분리 (모듈화)

DB 연결 로직을 `db.js` 로 분리해 전체 서버에서 재사용 가능하도록 설계. 유지보수 시 DB 설정 변경이 단 1개 파일만으로 가능.

서버단 중복 방지 로직

장바구니·위시리스트 추가 시 프론트가 아닌 서버단에서 DB 조회 후 검증. 클라이언트 조작에도 데이터 무결성 보장.

동적 카테고리 설계

카테고리를 하드코딩이 아닌 DB 기반으로 관리해 코드 수정 없이 카테고리 추가/삭제 가능. 가격 필터와 정렬도 동적 쿼리로 지원.

.env 보안 분리

DB 접속 정보를 `.env` 로 분리해 소스 코드에 직접 노출되지 않도록 보안 강화. 커넥션 풀로 DB 성능 최적화.

팀 프로젝트에서 배운 것

첫 팀 프로젝트였던 만큼 기술적인 어려움뿐 아니라 협업 자체의 어려움도 크게 느꼈습니다. 초반에는 각자 개발한 기능이 서로 맞지 않거나, 말로는 이해한 것 같았지만 실제 구현 결과가 달라 수정이 반복되는 상황이 생겼습니다. **의사소통이 충분하지 않으면 개발 속도보다 수정 비용이 더 커진다**는 것을 몸소 경험했습니다. 이 경험은 이후 KISETSU 솔로 프로젝트에서 설계를 철저히 하는 원동력이 되었습니다.

PROJECT 02

KISETSU

도매 패션 이커머스 플랫폼
실제 운영자 요구 기반 · 1인 풀스택 솔로 프로젝트

유형	기간	규모
1인 솔로 프로젝트	2026.01.15 – 01.30 (2주)	고객 기능 8개 + 관리자 기능 8개

React 19 · Node.js · Express 5 · MariaDB · JWT · Socket.IO · Chart.js · bcrypt · Context API

04

프로젝트 개요 · 아키텍처

KISETSU — 실제 도매업 운영자의 요구를 반영한 플랫폼

개발 목적

- 실제 도매업 운영자의 **온라인 판매 채널 부재 문제** 해결
- 재고·주문·고객 관리를 **하나의 플랫폼으로 통합**
- 오프라인 의존도를 낮추고 **운영 효율화** 도모
- 운영자(지인)와 직접 대화하며 요구사항 수집

주요 기능 목록

- 고객: 회원가입/로그인(JWT) · 상품목록 · 장바구니 → 주문
- 고객: 자동완성 검색 + 인기검색어
- 고객: 실시간 1:1 채팅 (Socket.IO)
- 고객: 계절별 테마 UI (봄/여름/가을/겨울)
- 관리자: 대시보드 · 상품/주문/회원 관리
- 관리자: Chart.js 매출 통계

시스템 아키텍처

Frontend →	React 19 + Vite	고객 화면	관리자 화면
통신 →	HTTP (Axios) REST API	WebSocket (Socket.IO)	
Backend →	Node.js + Express 5	/auth · /products · /orders · /admin · /chat	
Auth →	JWT 미들웨어	bcrypt 해싱	역할 기반 접근제어
Database →	MariaDB	커넥션 풀	트랜잭션 처리

05

주요 기능 구현

JWT 인증 · Socket.IO 채팅 · Context API · 인기검색어

JWT 인증 미들웨어

authMiddleware.js

```
const authMiddleware = (req, res, next) => {
  try {
    const token = req.headers
      .authorization?.split(" ")[1]
    if (!token)
      return res.status(401).json({
        msg: "인증 토큰 없음"
      })
    const decoded = jwt.verify(
      token, process.env.JWT_SECRET
    )
    req.user = decoded
    next()
  } catch {
    res.status(403).json({ msg: "유효하지 않은 토큰" })
  }
}
```

SOCKET.IO 실시간 채팅

Chat.jsx

```
// 1단계: 채팅 열릴 때 소켓 연결
useEffect(() => {
  if (isOpen && isLoggedIn) {
    const newSocket = io(SOCKET_URL)
    setSocket(newSocket)
    return () => newSocket.disconnect()
  }
}, [isOpen, isLoggedIn])

// 2단계: 소켓 생기면 이벤트 구독
useEffect(() => {
  if (!socket) return
  socket.on('message', (msg) => {
    setMessages(prev => [...prev, msg])
  })
  return () => socket.off('message')
}, [socket])
```

인기검색어 — SQL 비즈니스 로직

search.js

```
// 최근 7일 주문 기반 인기 상품 추출
const [popular] = await db.query(`
  SELECT p.name,
    COUNT(oi.order_item_id) as order_count
  FROM products p
  JOIN order_items oi ON p.id = oi.product_id
  JOIN orders o ON oi.order_id = o.id
  WHERE o.created_at ≥
    DATE_SUB(NOW(), INTERVAL 7 DAY)
  GROUP BY p.id
  ORDER BY order_count DESC
  LIMIT 5
`)
```

CONTEXT API — 장바구니 전역 상태

CartContext.jsx

```
const CartContext = createContext()

// 커스텀 훅으로 사용처 보호
export const useCart = () => {
  const ctx = useContext(CartContext)
  if (!ctx)
    throw new Error(
      "CartProvider 밖에서 호출됨"
    )
  return ctx
}

export const CartProvider = ({ children }) => {
  const [cart, setCart] = useState([])
  return (
    <CartContext.Provider value={{ cart, setCart }}>
      {children}
    </CartContext.Provider>
  )
}
```

06

결과 · 종합 회고

KISETSU 성과와 두 프로젝트를 통한 성장

8개	8개	100h	실전
고객 기능	관리자 기능	2주 단독 완성	운영자 요구 기반

잘된 점

- 첫 프로젝트 대비 코드 구조와 흐름이 읽히기 시작
- 기획부터 구현까지 전 과정을 혼자 완성
- 실제 운영자 요구를 반영한 서비스 설계 경험
- JWT, Socket.IO, Context API 등 실무 기술 적용
- 풀스택 데이터 흐름 전반 이해

아쉬운 점 / 다음 목표

- 테스트 코드 부재 — 다음 프로젝트에서 도입 목표
- 리팩토링 시간 부족으로 중복 코드 일부 존재
- TypeScript 도입으로 타입 안정성 확보 계획
- CI/CD 파이프라인 구성 학습 중

성장 스토리 — AuRa → KISETSU

AuRa 팀 프로젝트에서 협업의 어려움과 기술적 한계를 경험한 후, KISETSU에서는 **설계를 먼저, 구현은 그 다음**이라는 원칙을 철저히 지켰습니다. DB 테이블 설계, REST API 명세, 컴포넌트 구조를 사전에 정리한 덕분에 2주 안에 고객·관리자 기능 총 16개를 단독으로 완성할 수 있었습니다. 단순히 작동하는 코드를 넘어, **왜 이렇게 설계했는가를 설명할 수 있는 개발자**로 성장하는 것이 목표입니다.